

Simple and Effective Stochastic Neural Networks

Tianyuan Yu¹, Yongxin Yang¹, Da Li^{2,3}, Timothy Hospedales^{2,3}, Tao Xiang¹

¹Center for Vision, Speech and Signal Processing, University of Surrey

²School of Informatics, University of Edinburgh

³Samsung AI Centre, Cambridge

{tianyuan.yu, yongxin.yang, t.xiang}@surrey.ac.uk, da.li1@samsung.com, t.hospedales@ed.ac.uk

Abstract

Stochastic neural networks (SNNs) are currently topical, with several paradigms being actively investigated including dropout, Bayesian neural networks, variational information bottleneck (VIB) and noise regularized learning. These neural network variants impact several major considerations, including generalization, network compression, robustness against adversarial attack and label noise, and model calibration. However, many existing networks are complicated and expensive to train, and/or only address one or two of these practical considerations. In this paper we propose a simple and effective stochastic neural network (SE-SNN) architecture for discriminative learning by directly modeling activation uncertainty and encouraging high activation variability. Compared to existing SNNs, our SE-SNN is simpler to implement and faster to train, and produces state of the art results on network compression by pruning, adversarial defense, learning with label noise, and model calibration.

Introduction

Stochastic neural networks (SNNs) have a long history. Recently various stochastic neural network instantiations have been topical in their applications to reducing overfitting (Gal and Ghahramani 2016; Neelakantan et al. 2016) and training data requirements (Garnelo et al. 2018), providing confidence estimates on predictions (Gal and Ghahramani 2016), enabling network compression (Dai et al. 2018), improving robustness to adversarial attack (Aleml et al. 2017), improving optimization (Neelakantan et al. 2016), generative modeling (Kingma and Welling 2014), and inputting or producing probability distributions (de Bie, Peyré, and Cuturi 2019; Frogner, Farzaneh, and Solomon 2019).

One of the most theoretically appealing stochastic neural network formulations is Bayesian neural networks, which place a prior distribution on the network weights (Graves 2011; Blundell et al. 2015; Ritter, Botev, and Barber 2018). However this usually leads to more complex learning and inference procedures that rely on variational approximations or sampling. Another line of work instead focuses on modeling uncertainty in neural network activations. Notably the variational information bottleneck (VIB) (Aleml et al. 2017) is motivated by information theoretic considerations (Tishby,

Pereira, and Bialek 1999) to learn a hidden representation that carries maximum information about the output and minimum information about the input. Evaluating the required mutual information terms requires modeling probability distributions over activations rather than weights. Deep VIB (Aleml et al. 2017) leads to improved generalization, adversarial robustness and model compression algorithms (Dai et al. 2018). Furthermore, modeling noisy stochastic activations is often practically useful for improving exploration and local minima escape during optimization, generalization and adversarial robustness (You et al. 2019; Noh et al. 2017; Bishop 1995; Gulcehre et al. 2016), and in some cases can be linked back to Bayesian neural nets (Noh et al. 2017; Gal and Ghahramani 2016) when the noise added at each activation can be considered as a result of sampling from the weight posterior.

In this paper we propose a simple and effective stochastic neural network (SE-SNN) that models activation uncertainty through predicting a Gaussian mean and variance at each layer, which is then sampled during the forward pass. This is similar to the strategy used to model activation distributions in VAE (Kingma and Welling 2014) and VIB (Aleml et al. 2017). Differently, we then place a non-informative prior on the activation distribution and derive an activation regularizer that encourages high activation variability via preferring high-entropy activations. In conjunction with a discriminative learning loss, this means that the network is optimized for activation patterns that have high uncertainty while simultaneously being predictive of the target label. The interplay between these two objectives leads to several appealing capabilities in pruning, adversarial defense, learning with label noise, and improving model calibration. **Pruning:** Pruning aims to reduce the number of parameters in neural networks (NNs) while maintaining accuracy. Simultaneously optimizing for high per-activation variability/uncertainty and predictive accuracy leads to the network packing more entropy into the least significant neurons – so that the most crucial neurons are free to operate unperturbed. This leads to a simple pruning criterion based on each neuron’s entropy value. **Adversarial defense:** Adversarial defense methods aim to increase NNs resilience to adversarial attack. By optimizing for both per-activation uncertainty and the network’s predictive accuracy, a representation-level data augmentation policy is trained that perturbs the internal features during training for increased robustness (Aleml et al. 2017; You et al. 2019). **Label noise:**

Label noise is common in real-world datasets and tends to reduce learning performance. With SE-SNN, per-activation uncertainty can be easily aggregated to produce per-instance uncertainty. By optimizing for per-instance uncertainty and predictive accuracy, the network allocates the uncertainty the representation of the hard-to-classify instances so as to down-weight their influence on parameter learning. **Calibration:** In real-world decision-making, trustworthiness in the form of correctly calibrated confidence estimates, is often more critical than high accuracy – e.g., if the network is likely to make a mistake on a given instance, the decision can be delegated to a human. During inference, per-instance uncertainty can be considered as an indicator of confidence, and provide improved temperature scaling (Guo et al. 2017) to reduce calibration error.

To summarize, our contributions are: (1) A new simple yet effective stochastic neural network formulation. (2) We show that our SE-SNN has connections to VIB (Alemi et al. 2017), Dropout (Srivastava et al. 2014) and non-informative activation priors while being simpler to implement and faster to train, as well as impactful on a wider range of practical problems. (3) Comprehensive evaluations show excellent performance on pruning-based model compression, adversarial defense, label noise robust learning, and model calibration.

Related Work

Connection to VIB and Sparse VD Though we derive our max-entropy regularizer from the perspective of a non-informative activation prior, our work is closely related to VIB (Alemi et al. 2017) and sparse variational dropout (VD) (Molchanov, Ashukha, and Vetrov 2017), despite their different perspectives. Specifically, if we replace our infinite-variance Gaussian with a standard Gaussian, it becomes VIB (see Eq. 17 in (Alemi et al. 2017)). The max-entropy regularizer is also linked to Eq. 14 in Sparse VD (Molchanov, Ashukha, and Vetrov 2017), which also encourages large variance/entropy (at a different rate). But Sparse VD (Molchanov, Ashukha, and Vetrov 2017) is derived with a completely different motivation: It has an intuitive explanation that the regularizer corresponds to a sparsity prior on the weights. We note that enforcing uncertainty on activations rather than weights has a number of advantages: (i) The weight prior is intractable analytically, which leads to the fact that Sparse VD regularization is itself an approximation. (ii) Deriving the regularizer from a weight prior is unnecessarily complicated for the purpose of sparsifying the model. In contrast, our approach sidesteps the need to sample weights and avoids keeping multiple copies of the network, which can potentially improve efficiency (e.g., in memory usage).

Stochastic layers have been used in several other works in order to achieve better classification or regression accuracy. (Kingma, Salimans, and Welling 2015) proposes a generalization of Gaussian dropout where the dropout rates are learned, leading to higher classification accuracy. Natural-parameter networks (NPN) (Wang, Xingjian, and Yeung 2016) is a class of probabilistic neural networks where the input, target output, weights, and neurons can all be modeled by arbitrary exponential-family distributions (e.g., Poisson distribu-

tions for word counts) instead of being limited to Gaussian distributions, in order to help classification, regression, and representation learning tasks. To reduce computational cost, (Postels et al. 2019) approximates uncertainty estimates using a sampling-free approach and obtains better results on classification and regression tasks. Comparing to these, SE-SNN is again simpler and has wider-reaching impact.

Network Compression Network compression based on pruning typically uses heuristics based on pruning low-importance weights or low-activation neurons (Molchanov et al. 2017; Wen et al. 2016), often assisted by sparsity-enhancing priors such as lasso (Wen et al. 2016), log-uniform (Neklyudov et al. 2017), Jefferys and horse-shoe (Louizos, Ullrich, and Welling 2017). ℓ_0 (Louizos, Welling, and Kingma 2018) and ℓ_1 (Liu et al. 2017) norm-based regularizers get similar pruning effects. We avoid the complication of Bayesian learning of weights by proposing a simpler and direct activation prior that predisposes neurons towards deactivation unless necessary to solve the supervised task.

Adversarial Defense Our method is related to existing randomization-based defense methods (Xie et al. 2018; Alemi et al. 2017; Ye and Zhu 2018; Liu et al. 2019). However unlike these studies which use a fixed distribution for noise (Alemi et al. 2017), a learned model distribution for effective randomization (Liu et al. 2019), image perturbations (Xie et al. 2018) or a learned adversarial data-generating distribution (Ye and Zhu 2018), our randomization-based defense is both *learned*, and *data-dependent* since the variance at each layer is generated based on the output of the previous layer. The most relevant work to ours is RSE (Liu et al. 2018), which adds Gaussian noise layers to the NN to thwart gradient-based attacks. However, RSE uses a fixed constant noise variance. We introduce *learned* and *data-dependent* noise, which leads to improved performance.

Label Noise Robustness A number of existing label noise-robust deep learning approaches require a subset of noisy data to be reliably re-annotated (cleaned) to verify which samples contain noise (Lee et al. 2017; Jiang et al. 2018). In contrast, some others, similar to our SE-SNN, do not rely on additional human noise annotation. These methods address label noise by either iterative label correction via bootstrapping (Reed et al. 2015), adding additional layers on top of a softmax classification layer to estimate the noise pattern (Sukhbaatar et al. 2015; Goldberger and Ben-Reuven 2017), or loss correlation (Patrini et al. 2017). By allocating large uncertainty to outlying samples, our SE-SNN offers a simple and effective solution to learning with label noise. This approach to label noise robustness is appealingly simple in that it requires neither explicit detection of noisy samples, nor additional annotation. It is worth noting that label noise robustness has been largely ignored by existing SNNs such as VIB (Alemi et al. 2017) and sparse VD (Molchanov, Ashukha, and Vetrov 2017).

Calibration Modern neural networks for classification often suffer from poor calibration (Guo et al. 2017). I.e., the probabilities assigned to outputs are not reflective of the actual accuracy. While regularizers such as label-smoothing

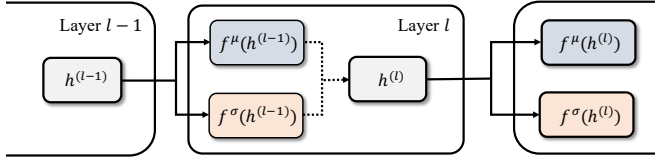


Figure 1: An illustration of the stochastic learning module in a SE-SNN. The output of layer l is sampled from the learned distribution defined by $f^\mu(h^{l-1})$ and $f^\sigma(h^{l-1})$.

(Müller, Kornblith, and Hinton 2019) improve calibration, they do not outperform calibrating the temperature of the output softmax based on a validation set (Guo et al. 2017). Dropout has also been used to model (Gal and Ghahramani 2016) uncertainty and improve predictive log-likelihood and RMSE. We show that the SE-SNN’s uncertainty estimate can be leveraged to improve existing model calibration techniques in (Guo et al. 2017; Gal and Ghahramani 2016).

Methodology

Stochastic Layers We consider a neural network discriminatively trained for a predictive task such as object recognition. Instead of computing fixed point estimates of feature vectors, we propose to use *stochastic* neurons. More specifically, for an input h , a layer will output a series of univariate distributions. By sampling from those distributions independently, we get a random output z . Finally we apply the non-linear activation function $\psi(\cdot)$ to z and get the input for the next layer. In this study, we choose to use Gaussian distribution with parameterized mean and variance, which has been popularized by VAE (Kingma and Welling 2014) and VIB (Alemi et al. 2017) due to the ease of reparameterization. Formally, for the l -th layer, this forward-pass process can be written as (omitting neuron index for notation simplicity),

$$\begin{aligned}\mu^{(l)} &= f^\mu(h^{(l-1)}), \\ \sigma^{(l)} &= f^\sigma(h^{(l-1)}), \\ z^{(l)} &\sim \mathcal{N}(\mu^{(l)}, \sigma^{2(l)}), \\ h^{(l)} &= \psi(z^{(l)}).\end{aligned}\quad (1)$$

The above process is illustrated in Figure 1, where each standard deviation predictor f^σ uses softplus activation $f(x) = \log(1 + \exp(x))$ to ensure positivity.

Supervised Learning Loss We can choose to replace some or all intermediate layers of a vanilla neural network with such stochastic layers. For the final layer (i.e., the classifier) we opt for a standard linear layer, and the classification loss L_{cl} is the same as that of a vanilla neural network, e.g., cross-entropy. Since a Gaussian distribution is fully reparameterizable, the network can be trained end-to-end as long as the sampling process $z \sim \mathcal{N}(\mu, \sigma^2)$ is realized by $z = \mu + \epsilon \cdot \sigma$ where $\epsilon \sim \mathcal{N}(0, 1)$.

Max-entropy Regularization We place a non-informative prior on the produced Gaussian (denoted as $\mathcal{N}(\mu_2, \sigma_2^2)$). The non-informative prior is a Gaussian with arbitrary mean (μ_2) and infinite variance (σ_2^2). This reflects

the prior that none of neurons is meaningful for predictive purposes. We minimize the KL divergence of the produced Gaussian from the prior Gaussian,

$$\begin{aligned}& \min_{\mu_1, \sigma_1} \left(\lim_{\sigma_2 \rightarrow \infty} \text{KL}(\mathcal{N}(\mu_1, \sigma_1^2) \parallel \mathcal{N}(\mu_2, \sigma_2^2)) \right) \\ & \Rightarrow \min_{\mu_1, \sigma_1} \left(\lim_{\sigma_2 \rightarrow \infty} \left(\log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} \right) \right) \\ & \Rightarrow \min_{\sigma_1} \left(\lim_{\sigma_2 \rightarrow \infty} \left(\log \frac{\sigma_2}{\sigma_1} \right) \right) \Rightarrow \min_{\sigma_1} (-\log \sigma_1).\end{aligned}\quad (2)$$

Eq. 2 suggests that we simply need to maximize the predicted standard deviation, or equivalently the entropy of the predicted Gaussian. Thus we call it a *max-entropy regularizer* Ω . It can be easily used in any existing neural network architecture:

$$\min_{\theta} -\log(\sigma(h|\theta)), \quad (3)$$

where $\sigma(h|\theta)$ denotes the predicted standard deviation of hidden unit h given the neuron uncertainty prediction parameter θ . For numerical safety, we introduce a margin b in the loss,

$$\min_{\theta} (b - \log(\sigma(h|\theta)))^+. \quad (4)$$

This means that the regularization does not punish the model as long as the entropy is larger than a threshold b . Note that, this loss design is not absolutely necessary as the increment for σ shrinks (since the gradient is $-\frac{1}{\sigma}$) during training, and thus never reaches infinity with a finite number of updates. However, one can think of it as an early-stopping mechanism for this regularization term.

So far we have introduced the stochasticity to the smallest unit of a network, i.e., a single neuron. To compute the value of the regularizer over a mini-batch consisting of N training samples, we need to aggregate the entropy of multiple neurons and set the margin b on the aggregation. How to aggregate exactly is task-dependent, and we next provide some suggestions for the four tasks to be discussed next, including network pruning, adversarial defense, label noise defense, and model calibration.

Pruning For network pruning, we aggregate entropy over samples for each neuron, and then penalize if that neuron’s aggregated entropy is low. To this end, the regularizer is formulated as:

$$\Omega(\theta) = \frac{1}{K} \sum_{j=1}^K \left(b - \frac{1}{N} \sum_{i=1}^N \log(\sigma_{i,j}) \right)^+, \quad (5)$$

where $i = 1 \dots N$ indexes the training samples, and $j = 1 \dots K$ neurons in a layer (i.e., feature channels). This regularizer aims to make neurons very stochastic, to the point of compromising their reliability for computing a supervised learning task. Thus only those neurons that are most useful for the task get their entropy lowered and thus pay the regularization cost. Less useful neurons get their entropy maximized, allowing them to be detected and pruned after training.

Label Noise Different from pruning, we aim to identify uncertain samples. Therefore, we aggregate entropy over neurons for each sample, and prefer high sample-wise entropy,